여행 상품 구매 예측

고객 정보를 바탕으로 여행 상품 구매 여부를 예측하는 파이프라인을 정리했습니다.

원본 데이터는 **4,888행 · 22컬럼**이며, 이 중 `CustomerID`와 정답 컬럼 `ProdTaken`을 제외해 **20개 입력 변수**로 학습했습니다.

전처리 · 피처 엔지니어링 · 모델 비교 · 하이퍼파라미터 튜닝 · 최종 모델 해석

팀원: 김준수 · 김한별 · 서동열 | **데이터 출처**: Kaggle - Holiday Package Purchase Prediction

1단계. 데이터 불러오기

제공된 `Travel.csv` 파일을 불러와 데이터 구조를 확인했습니다.

데이터 출처

Kaggle의 Holiday Package Purchase Prediction 데이터를 사용했습니다.

입력 변수 구성

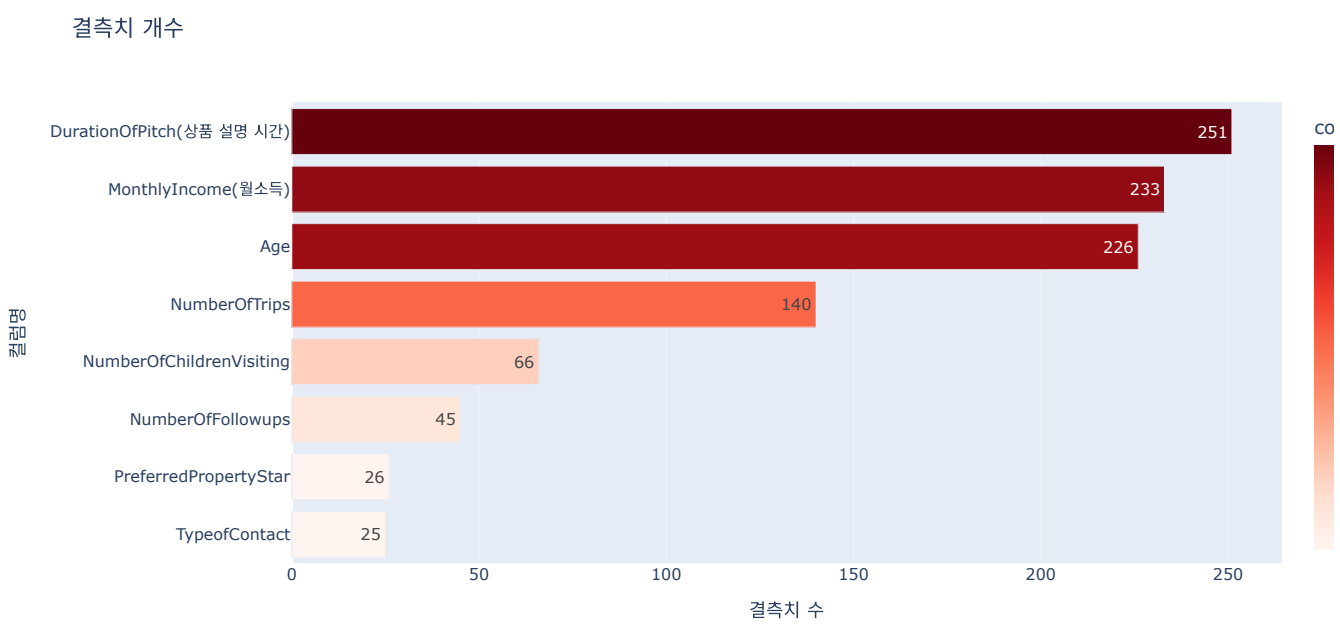
고객 식별용 컬럼인 `CustomerID`와 정답 컬럼 `ProdTaken`은 제외하고 학습했습니다.

코드 화면 · 데이터 불러오기

```
# 구글드라이브 마운트
from google.colab import drive
drive.mount('/content/drive')

# CSV 파일 불러오기
df = pd.read_csv('/content/drive/MyDrive/Travel.csv')

# 데이터 크기와 샘플 확인
print(f'데이터 크기: {df.shape[0]}명 x {df.shape[1]}개 변수')
display(df.head(3))
```

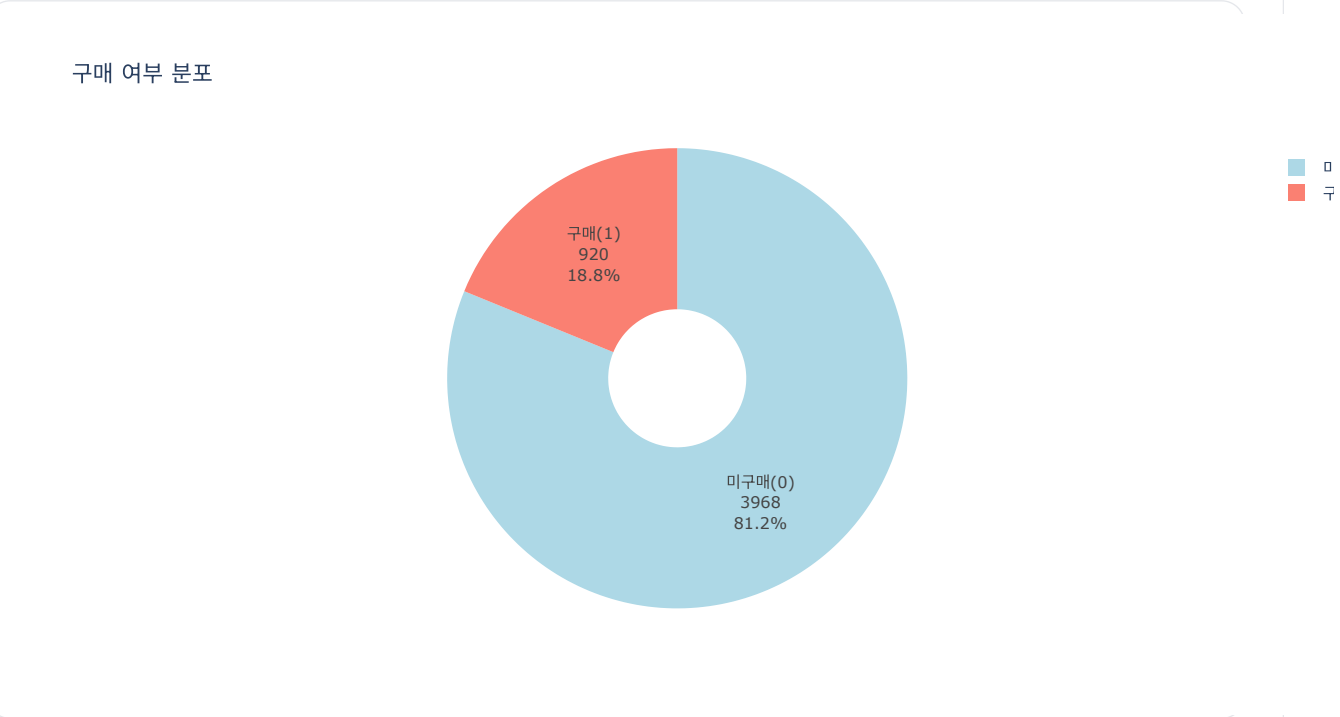
2단계. 데이터 살펴보기 (EDA)

결측치, 구매 비율, 직업별 구매율을 시각화해 데이터 특징을 먼저 확인했습니다.



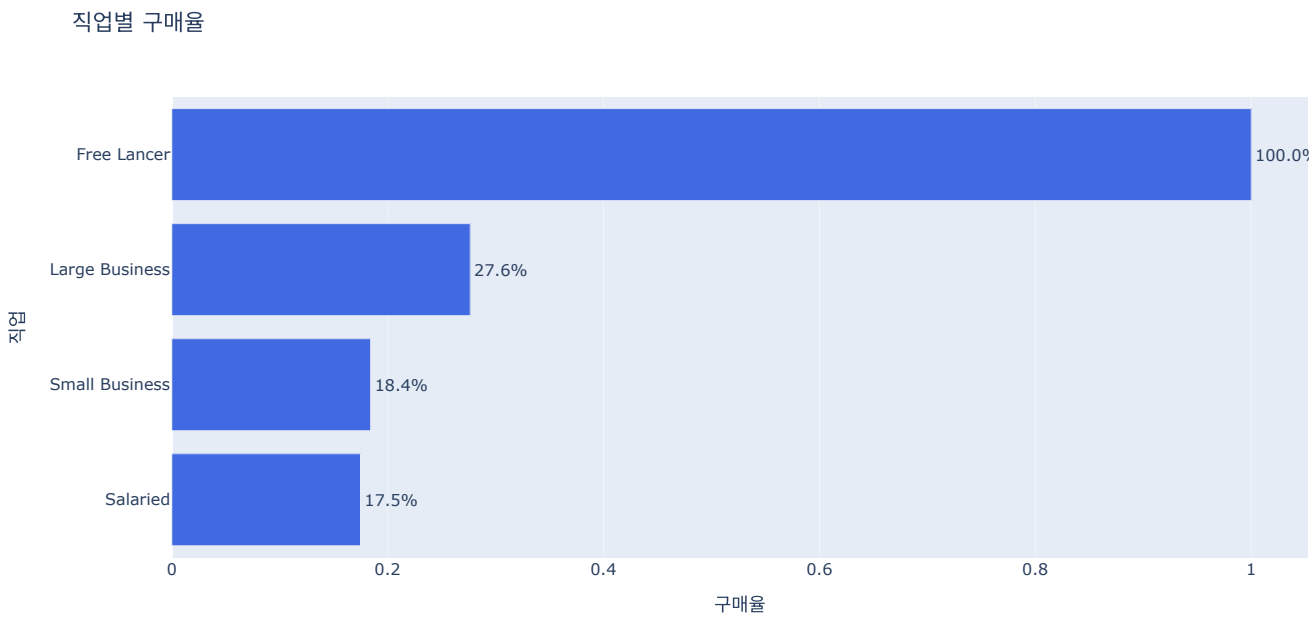
결측치 그래프 해석

이 그래프는 비어 있는 값이 있는 컬럼과 그 개수를 보여줍니다. DurationOfPitch(상품 설명 시간), MonthlyIncome(월소득), Age(나이) 등 여러 변수에 비어 있는 값이 있어서, 모델 학습 전에 결측치 처리가 필요하다고 판단했습니다.



구매 여부 분포 해석

구매 고객은 약 18.8%, 미구매 고객은 약 81.2%로 확인되었습니다. 즉 타깃 값이 한쪽으로 치우쳐 있어, 이후 모델 평가에서는 정확도(Accuracy)뿐 아니라 F1 점수(F1 Score)와 ROC-AUC(전반적인 분류 성능)도 함께 보았습니다.



직업별 구매율 해석

직업별 평균 구매율에 차이가 있어 Occupation도 예측에 도움이 될 수 있는 변수라고 보았습니다. 특히 Free Lancer는 표본 수가 2명뿐이고 그 2명이 모두 구매해 100%로 나타난 것이므로, 일반화하기보다는 참고용 지표로 해석했습니다.

코드 화면 · EDA

```
# 결측치 개수 확인
missing = df.isnull().sum()
missing = missing[missing > 0].sort_values(ascending=True)

# 막대형 그래프 : 컬럼별 결측치 개수
px.bar(x=missing.values, y=missing.index, orientation='h',
       title='결측치 개수', text=missing.values,
       color=missing.values, color_continuous_scale='Reds',
       labels={'x':'결측치 수','y':'컬럼명'}).show()

# 구매 여부 분포 확인
cnt = df['ProdTaken'].value_counts().rename(index={0:'미구매(0)',1:'구매(1)'})
fig2 = px.pie(names=cnt.index, values=cnt.values, title='구매 여부 분포',
              color_discrete_sequence=['lightblue','salmon'], hole=0.3)
fig2.update_traces(textinfo='label+percent+value')
fig2.show()

# 직업별 평균 구매율 계산
occ = df.groupby('Occupation(직업)')['ProdTaken'].mean().sort_values().reset_index()
occ.columns = ['직업','구매율']
fig3 = px.bar(occ, x='구매율', y='직업', orientation='h',
              title='직업별 구매율',
              text=[f'{v:.1%}' for v in occ['구매율']],
              color_discrete_sequence=['royalblue'])
fig3.update_traces(textposition='outside')
fig3.show()
```

결측치가 존재하는 컬럼을 먼저 확인했고, 구매 여부 비율과 직업별 구매율을 보면서 어떤 변수가 예측에 도움이 될지 감을 잡았습니다.

3단계. 전처리 및 피처 엔지니어링

모델 학습 전에 결측치 처리, 인코딩, 스케일링을 적용하고, 새 변수를 만들어 예측 정보를 보강했습니다.

전처리

- 숫자형: 중앙값 대체 → 표준화
- 범주형: 최빈값 대체 → 원-핫 인코딩

파생 변수

- `TotalVisitors`: 총 방문 인원
- `HighIncome`: 고소득 여부

코드 화면 · 전처리 & 피처 엔지니어링

```
# 파생 변수 추가 (원래 없던 컬럼)
df['NumberOfChildrenVisiting'] = df['NumberOfChildrenVisiting'].fillna(0)  # 동반 자녀 수가 비어 있으면 0으로 채움
df['TotalVisitors'] = df['NumberOfPersonVisiting'] + df['NumberOfChildrenVisiting']  # 총 방문 인원 수 변수 생성
df['HighIncome'] = (df['MonthlyIncome(월소득)'] >= df['MonthlyIncome(월소득)'].median()).astype(int)  # 월소득이 중앙값 이상이면 1, 아니면 0

# 입력 변수 X와 정답(label) y를 나눔
X = df.drop(columns=['CustomerID','ProdTaken'])
y = df['ProdTaken']

# 데이터를 학습용(80%)과 테스트용(20%)으로 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 숫자형 컬럼과 범주형 컬럼 분리
num_cols = X_train.select_dtypes(include='number').columns.tolist()
cat_cols = X_train.select_dtypes(exclude='number').columns.tolist()

# 숫자형: 빈 값 -> 중앙값 -> 표준화
num_pipe = Pipeline([('imputer', SimpleImputer(strategy='median')),
                     ('scaler', StandardScaler())])
# 범주형: 빈 값 -> 최빈값 -> 원-핫 인코딩
cat_pipe = Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
                     ('onehot', OneHotEncoder(handle_unknown='ignore'))])
preprocessor = ColumnTransformer([('num', num_pipe, num_cols),
                                  ('cat', cat_pipe, cat_cols)])
```

4단계. 3가지 모델 학습 및 성능 비교

전처리 파이프라인은 공통으로 사용하고, 모델만 교체해 성능을 비교했습니다.

로지스틱 회귀

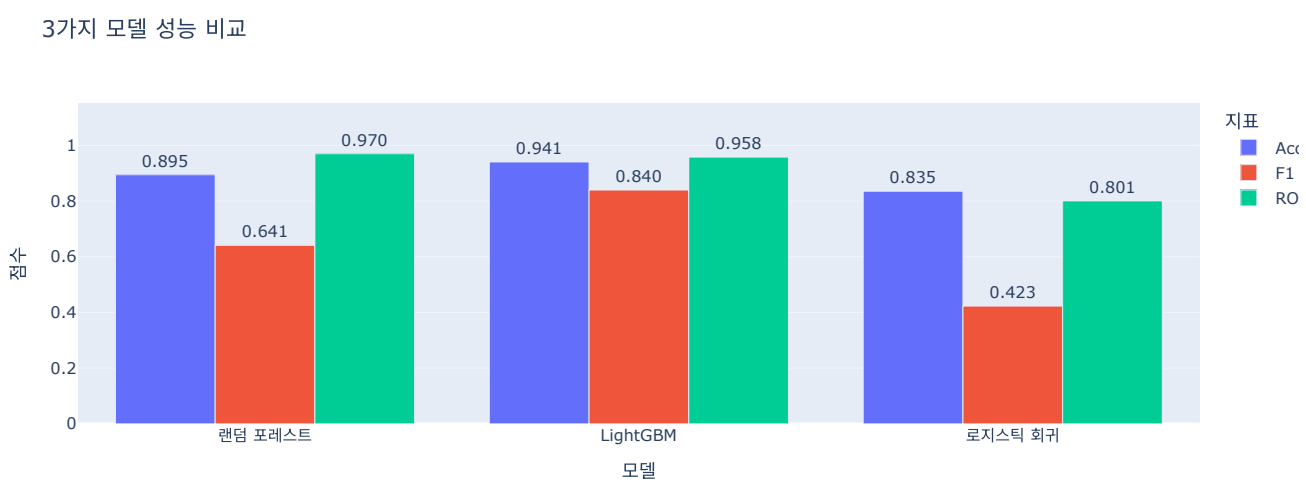
기본 비교 모델

랜덤 포레스트

안정적인 트리 기반 앙상블

LightGBM

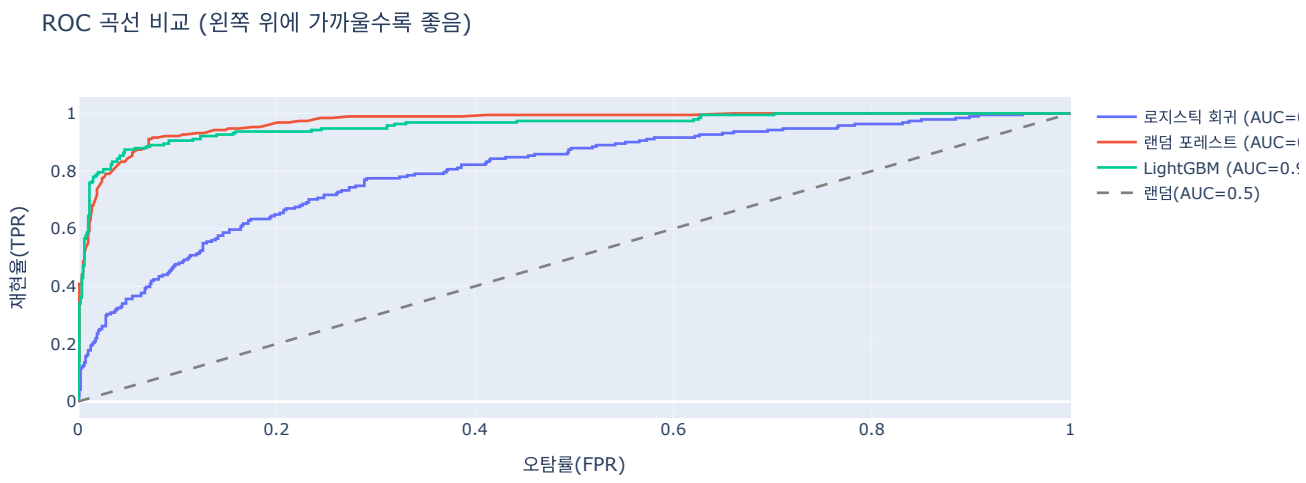
빠르고 성능이 좋은 부스팅 모델



코드 화면 · 모델 비교

```
# 비교할 모델을 설정
models_config = {
    '로지스틱 회귀': LogisticRegression(max_iter=1000, random_state=42),  # 기본 비교 모델
    '랜덤 포레스트': RandomForestClassifier(n_estimators=200, class_weight='balanced', random_state=42),  # 안정적인 트리 기반
앙상블
    'LightGBM': LGBMClassifier(n_estimators=400, learning_rate=0.05,
                               class_weight='balanced', random_state=42, verbose=-1)  # 빠르고 성능이 좋은 부스팅 모델
}

# 결과 저장용 리스트
results = []

# 모델별 학습 및 예측
for name, model in models_config.items():
    pipe = Pipeline([('preprocessor', preprocessor), ('model', model)])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    y_proba = pipe.predict_proba(X_test)[:,1]
```

막대그래프 기준으로 LightGBM은 Accuracy와 F1 Score가 가장 높았고, 랜덤 포레스트는 ROC-AUC가 가장 높았습니다. 로지스틱 회귀는 전체적으로 가장 낮은 성능을 보였습니다.



ROC 곡선에서도 세 모델 모두 랜덤 기준선보다 높은 성능을 보였고, 그중 랜덤 포레스트가 가장 높은 AUC를 기록했습니다. 다만 정확도와 F1 Score까지 함께 보면 LightGBM이 더 균형 잡힌 모델로 보였습니다.

5단계. 하이퍼파라미터 튜닝

기본 모델 비교 결과를 바탕으로 LightGBM에 대해 `RandomizedSearchCV`로 튜닝을 진행했습니다.

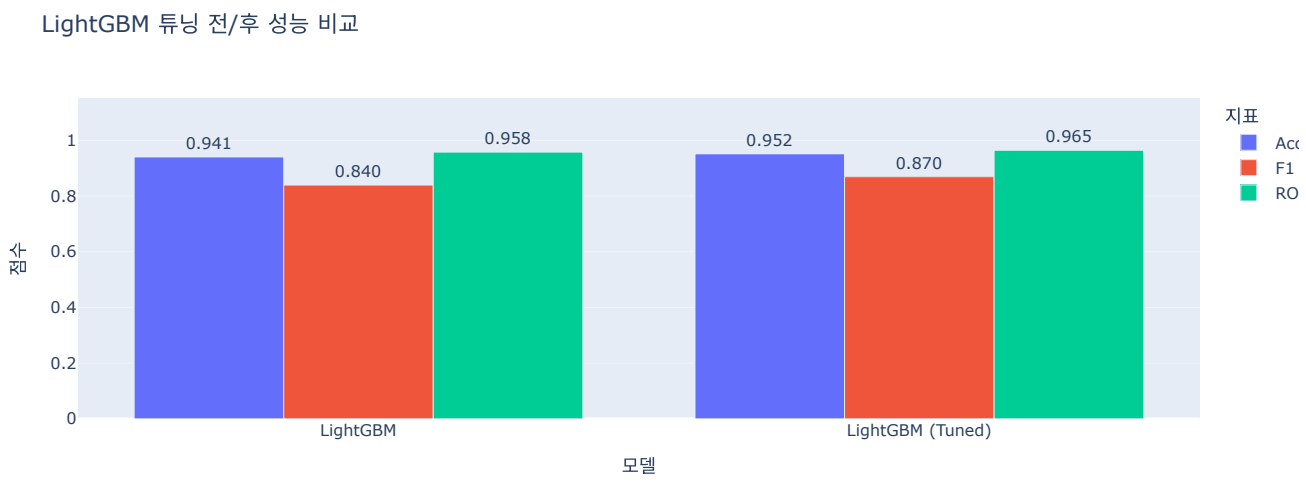
탐색 파라미터

`n_estimators`, `learning_rate`, `num_leaves`

검증 방식

3-Fold 교차검증, ROC-AUC 기준 선택

	모델	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	LightGBM (Tuned)	0.9519	0.9235	0.822	0.8698	0.9645



코드 화면 · 하이퍼파라미터 튜닝

```
# LGBM 하이퍼파라미터 튜닝
# RandomizedSearchCV는 여러 하이퍼파라미터 조합을 랜덤하게 시험해보고,
# 교차검증을 통해 가장 좋은 조합을 찾는 방법이다.
from sklearn.model_selection import RandomizedSearchCV

# 탐색할 하이퍼파라미터 후보 설정
param_dist = {
    'model__n_estimators': [200, 300, 400],      # 트리 개수
    'model__learning_rate': [0.03, 0.05, 0.1],   # 학습률
    'model__num_leaves': [15, 31, 63]            # 리프 노드 수
}

# 전처리 + LightGBM 모델 파이프라인 구성
lgb_pipe = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LGBMClassifier(class_weight='balanced', random_state=42, verbose=-1))
])

# RandomizedSearchCV로 최적의 파라미터 탐색
search = RandomizedSearchCV(
    lgb_pipe,
    param_distributions=param_dist,
    n_iter=5,
    scoring='roc_auc',
    cv=3,
    random_state=42
)
search.fit(X_train, y_train)
```

튜닝 후 LightGBM은 정확도(Accuracy), F1 점수(F1 Score), 재현율(Recall)이 모두 개선되었습니다. ROC-AUC(전반적인 분류 성능)는 랜덤 포레스트보다 약간 낮지만, 실제 구매 고객을 놓치지 않는 측면까지 고려했을 때 가장 균형 잡힌 성능을 보였습니다.

6단계. 혼동행렬(Confusion Matrix)

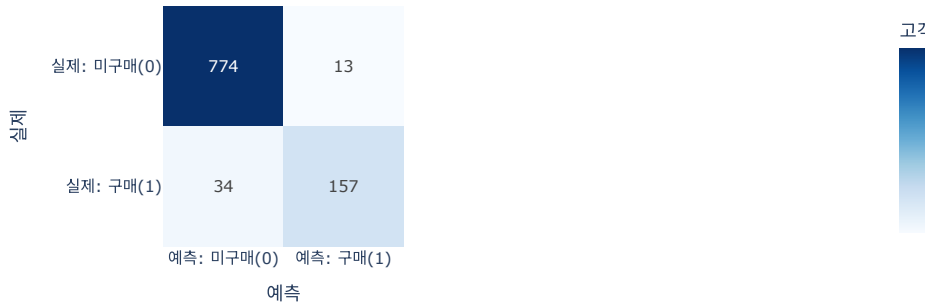
최종 선택 모델인 **LightGBM (Tuned)**의 예측을 실제 값과 비교했습니다.

해석 기준

- TN: 미구매 고객을 정확히 미구매로 예측
- FP: 미구매 고객인데 구매로 잘못 예측
- FN: 실제 구매 고객을 놓침
- TP: 구매 고객을 정확히 구매로 예측

주요 성능

```
최종 선택 모델: LightGBM (Tuned)
ROC-AUC: 0.9645
F1 Score: 0.8698
TN=774: 미구매 -> 정확히 예측
FP=13: 미구매인데 구매로 잘못 예측
FN=34: 구매할 고객을 미구매로 놓침
TP=157: 구매 -> 정확히 예측
```



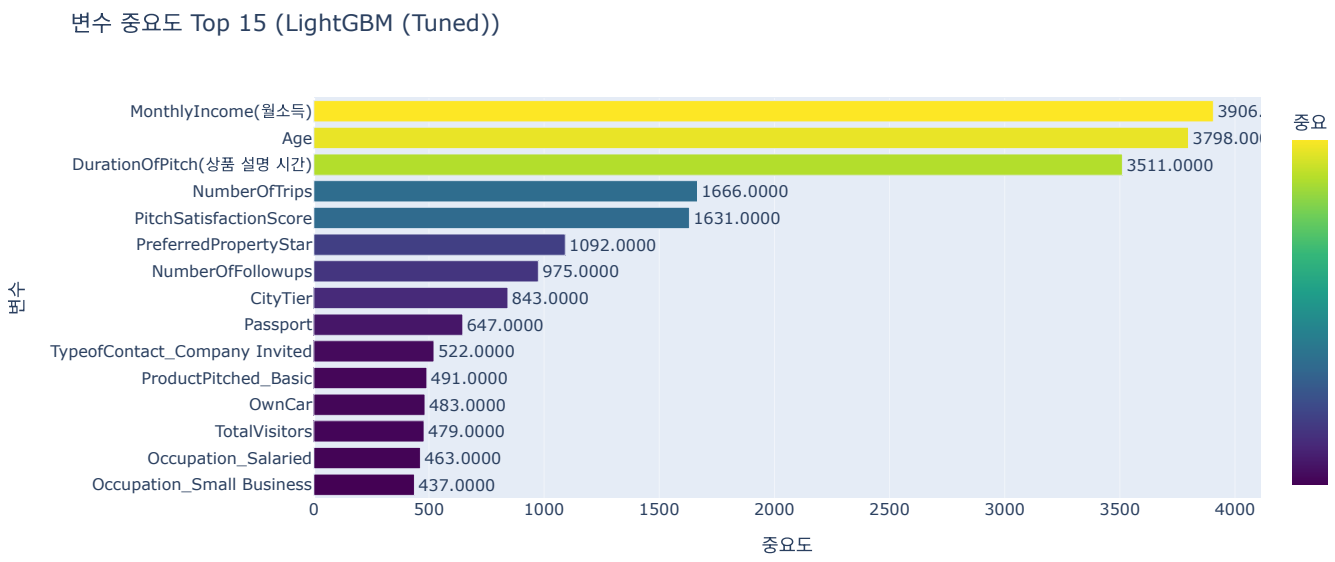
코드 화면 · 혼동행렬

```
# 최종 선택한 모델 확인
best_name = 'LightGBM (Tuned)'
best_pipe = search.best_estimator_

# 선택한 모델로 예측값 / 예측 확률 생성
y_pred_b = best_pipe.predict(X_test)
y_proba_b = best_pipe.predict_proba(X_test)[:, 1]

# 혼동행렬 계산
cm = confusion_matrix(y_test, y_pred_b)

# 혼동행렬 시각화
px.imshow(
    cm,
    labels=dict(x='예측', y='실제', color='고객 수'),
    x=['예측: 미구매(0)', '예측: 구매(1)'],
    y=['실제: 미구매(0)', '실제: 구매(1)'],
    title=f'혼동 행렬 - {best_name}',
    color_continuous_scale='Blues',
    text_auto=True
).update_layout(height=360).show()
```

혼동행렬 결과를 보면 미구매 고객과 구매 고객을 전반적으로 안정적으로 구분하고 있습니다. 특히 구매 고객을 놓치는 수가 크게 많지 않아 최종 모델로서 활용 가치가 높다고 해석할 수 있습니다.

7단계. 최종 모델 변수 중요도 분석

튜닝된 LightGBM이 어떤 변수를 중요하게 판단했는지 확인했습니다.



코드 화면 · 변수 중요도

```
# 최종 모델과 전처리기 불러오기
clf = best_pipe.named_steps['model']
pre = best_pipe.named_steps['preprocessor']

# 전처리 후 변수 이름 가져오기
feat_names = pre.get_feature_names_out()

# 변수 중요도 데이터프레임 만들기
imp_df = pd.DataFrame({'변수': feat_names, '중요도': clf.feature_importances_})

# 변수 이름 정리
imp_df['변수'] = imp_df['변수'].str.replace('num__', '').str.replace('cat__', '')

# 중요도가 높은 상위 15개 변수만 선택
imp_df = imp_df.nlargest(15, '중요도').sort_values('중요도')

# 가장 중요한 변수 확인
top_var = imp_df.iloc[-1]['변수']
print(f'예측에 가장 큰 영향을 준 변수: {top_var}')
```

변수 중요도가 높을수록 예측에 더 큰 영향을 준다고 볼 수 있습니다. 최종 그래프 기준으로 MonthlyIncome(월소득, 3906), Age(나이, 3798), DurationOfPitch(상품 설명 시간, 3511)가 특히 중요한 변수로 나타났습니다. 즉, 최종 모델은 고객의 월소득, 나이, 상품 설명 시간을 중요한 판단 기준으로 사용한 것으로 해석할 수 있습니다.

```
예측에 가장 큰 영향을 준 변수: MonthlyIncome(월소득)
```

8단계. 모델 저장

최종 모델을 파일로 저장해두면, 다음에 다시 학습하지 않고 바로 불러와 사용할 수 있습니다.

```
저장 완료: travel_best_model.pkl
불러오기 완료
ROC-AUC: 0.9645 (<- 위에서 나온 결과와 동일하면 정상)
```

코드 화면 · 모델 저장

```
# 최종 모델 저장 경로 설정
save_path = 'travel_best_model.pkl'

# 학습한 모델 저장
with open(save_path, 'wb') as f:
    pickle.dump(best_pipe, f)

# 저장한 모델 다시 불러오기
with open(save_path, 'rb') as f:
    loaded_model = pickle.load(f)

# 불러온 모델의 성능 확인
auc_check = roc_auc_score(y_test, loaded_model.predict_proba(X_test)[:,1])
print(f'ROC-AUC: {auc_check:.4f} (이전 결과와 동일하면 정상)')
```

> 저장한 모델을 다시 불러온 뒤에도 ROC-AUC 값이 동일하게 유지되어, 모델이 정상적으로 저장되고 재사용 가능함을 확인했습니다.